VISUALIZATION LIBRARY DOCUMENTATION

Matplotlib and Seaborn :

ShadowFax

Submitted By :

Anu bharti

Table of Contents :

Introduction to Data Visualization :

Data visualization is the process of representing data in a graphical form so that it becomes easier to understand. Instead of looking at large tables of numbers, graphs and charts help us identify patterns, trends, and relationships within the data.

Nowadays, data visualization is used in almost every field such as business, education, healthcare, finance, and research. It helps people make better decisions by presenting information in a simple and attractive way.

Python provides many libraries for creating different types of visualizations. Some of the most popular visualization libraries are:

- Matplotlib
- Seaborn
- Plotly
- Bokeh
- Pandas Visualization

In this documentation, we will learn about **Matplotlib** and **Seaborn**, which are two of the most commonly used visualization libraries in Python.

Matplotlib :

Matplotlib is one of the oldest and most popular data visualization libraries in Python. It was created by **John Hunter** in 2002. It is built on top of the **NumPy** library and is mainly used to create static, animated, and interactive graphs.

Matplotlib allows users to create simple as well as highly customized charts. It is widely used by students, researchers, and data analysts because of its flexibility.

Features of Matplotlib

Some important features of Matplotlib are:

- Easy to learn and use.
- Supports a large variety of graphs.
- Highly customizable.
- Works well with NumPy and Pandas.
- Can create publication-quality figures.
- Free and open-source library.
- Available on Windows, Linux, and macOS.

Installation :

Before using Matplotlib, it must be installed.

```
pip install matplotlib
```

If you are using Google Colab, Matplotlib is already installed.

Importing Matplotlib :

The pyplot module is used for creating graphs.

```
import matplotlib.pyplot as plt
```

If numerical data is required, NumPy can also be imported.

```
import numpy as np
```

Components of Matplotlib

Before creating graphs, it is important to understand the basic components of Matplotlib.

1. Figure

A Figure is the main window that contains one or more plots. Every graph created in Matplotlib is placed inside a figure.

Example:

```
fig = plt.figure()
```

2. Axes

Axes represent the plotting area where the graph is drawn. A figure can contain one or multiple axes.

The x-axis represents horizontal values, while the y-axis represents vertical values.

3. Axis

An Axis controls the scale, tick marks, and labels shown on the graph.

For example:

```
plt.xlabel("Months")
```

```
plt.ylabel("Sales")
```

4. Title

A title helps users understand what the graph represents.

Example:

```
plt.title("Monthly Sales")
```

5. Legend

A legend is used when more than one dataset is displayed in the same graph.

Example:

```
plt.legend()
```

Common Graphs in Matplotlib

Matplotlib supports many types of graphs. Some of the commonly used graphs are:

- Line Plot
- Scatter Plot
- Bar Chart
- Histogram
- Pie Chart
- Box Plot

Each graph will be discussed in detail in the next section with its description, use case, code example, output, and explanation

Why Use Matplotlib?

Matplotlib is useful because:

- It provides complete control over graphs.
- It supports many chart types.
- It is widely used in data science and machine learning.
- It produces high-quality visualizations suitable for reports and presentations.
- It is supported by a large Python community.

1. Line Plot

Description

A line plot is one of the simplest and most commonly used graphs in Matplotlib. It joins different data points with a continuous line. It is mainly used to show how values change over time.

Use Case

- Monthly sales report
- Temperature changes
- Stock market prices
- Population growth

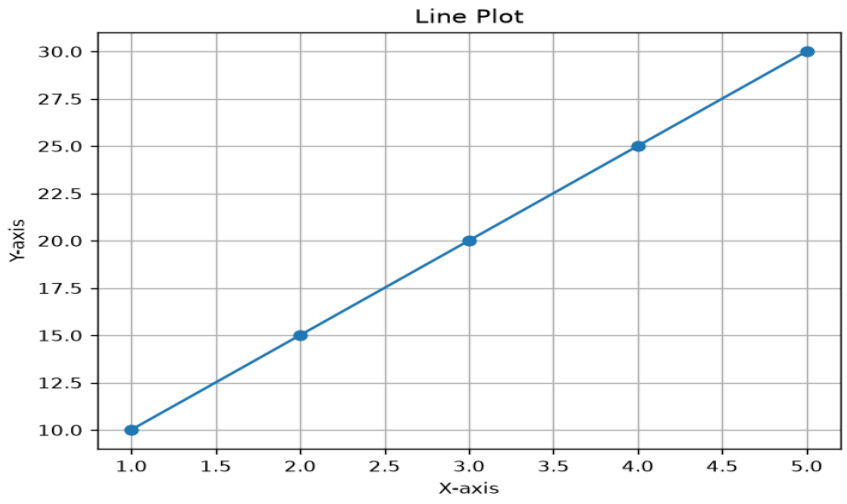
Code Snippet:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 15, 20, 25, 30]

plt.plot(x, y, marker='o')
plt.title("Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()
```

Output:



Explaination:

In the above program, the plot() function is used to create a line graph. The marker='o' parameter displays a small circle on each data point. The title(), xlabel(), and ylabel() functions are used to add a title and labels to the graph. Finally, show() displays the graph on the screen.

2. Bar Chart

Description

A bar chart uses rectangular bars to compare different categories. The height of each bar represents the value of that category.

Use Case

- Comparing students' marks
- Sales of different products
- Population of different cities

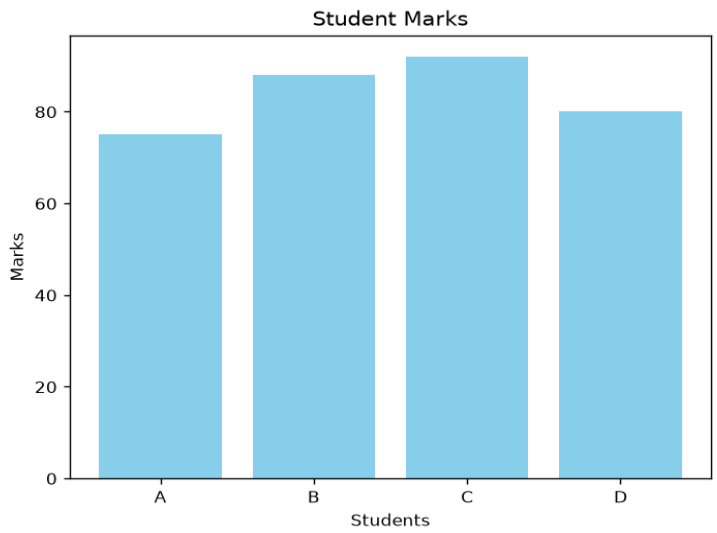
Code Snippet:

```
import matplotlib.pyplot as plt

students = ["A", "B", "C", "D"]
marks = [75, 88, 92, 80]

plt.bar(students, marks, color="skyblue")
plt.title("Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.show()
```

Output:



Explanation

The bar() function is used to create a bar chart. The x-axis contains the names of the students, while the y-axis contains their marks. Different colors can also be used to make the graph more attractive.

3. Histogram

Description

A histogram shows how data is distributed. It groups similar values into different intervals called bins.

Use Case

- Exam score distribution
- Age distribution
- Salary analysis

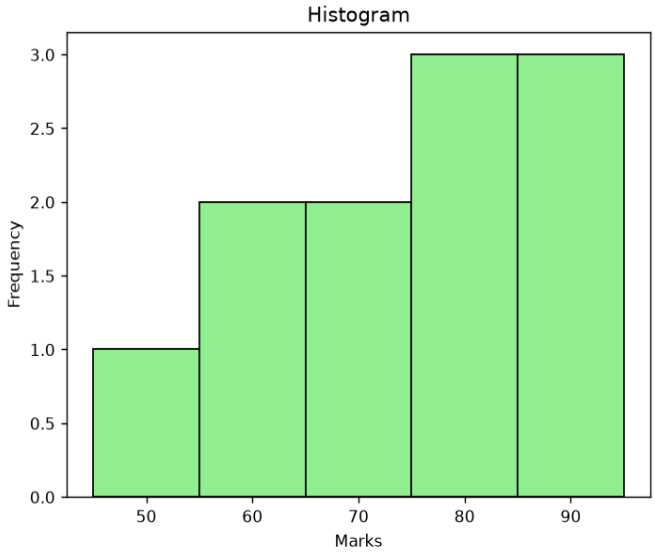
Code Snippet:

```python
import matplotlib.pyplot as plt

marks = [45, 55, 60, 65, 70, 75, 80, 82, 85, 90, 95]

plt.hist(marks, bins=5, color="lightgreen", edgecolor="black")
plt.title("Histogram")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Output:



Explanation:

The hist() function creates the histogram. The bins parameter divides the data into different groups. The edgecolor parameter makes the borders of each bar clearly visible.

4. Scatter Plot

Description

A scatter plot displays individual data points on a graph. It is mainly used to study the relationship between two variables.

Use Case

- Height vs Weight
- Study Hours vs Marks
- Advertisement Cost vs Sales

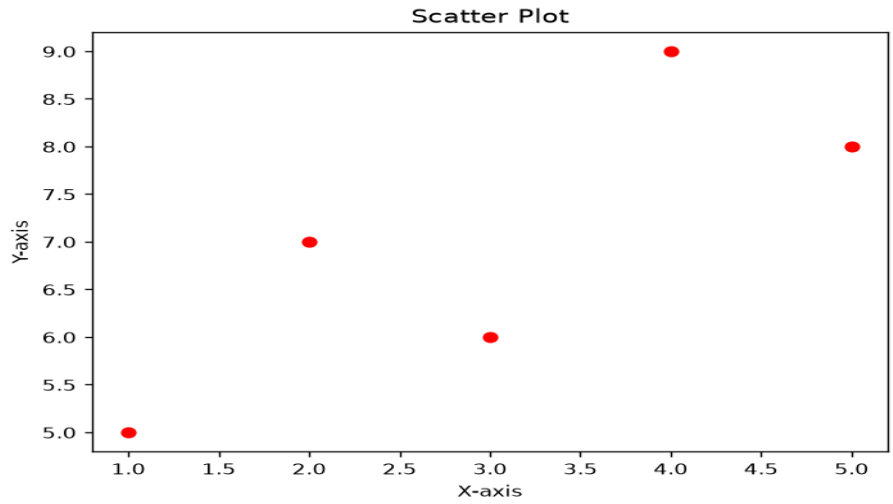
Code Snippet:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [5, 7, 6, 9, 8]

plt.scatter(x, y, color="red")
plt.title("Scatter Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

Output:



Explanation:

The scatter() function plots individual points instead of connecting them with lines. Scatter plots are useful for finding trends or relationships between two sets of data.

5. Pie Chart

Description

A pie chart represents data as slices of a circle. Each slice shows the percentage contribution of a category.

Use Case

- **Budget distribution**
- **Market share**
- **Survey results**

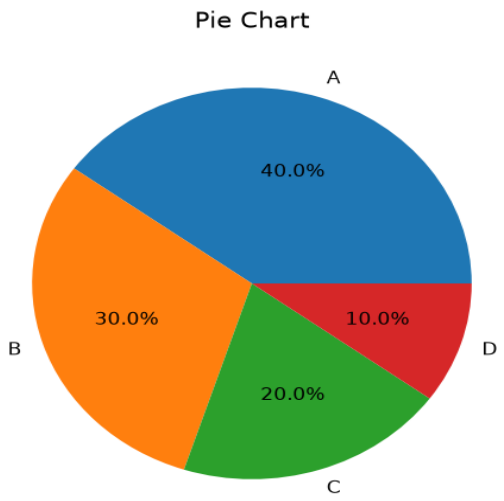
Code Snippet:

```
import matplotlib.pyplot as plt

sizes = [40, 30, 20, 10]
labels = ["A", "B", "C", "D"]

plt.pie(sizes, labels=labels, autopct="%1.1f%%")
plt.title("Pie Chart")
plt.show()
```

Output:



Explanation:

The pie() function is used to create a pie chart. The labels parameter displays the category names, while autopct automatically shows the percentage of each slice.

6. Box Plot

Description

A box plot is used to understand the spread of data and identify outliers. It shows the minimum value, maximum value, median, and quartiles of the dataset.

Use Case

- **Salary analysis**
- **Exam marks**
- **Data distribution**

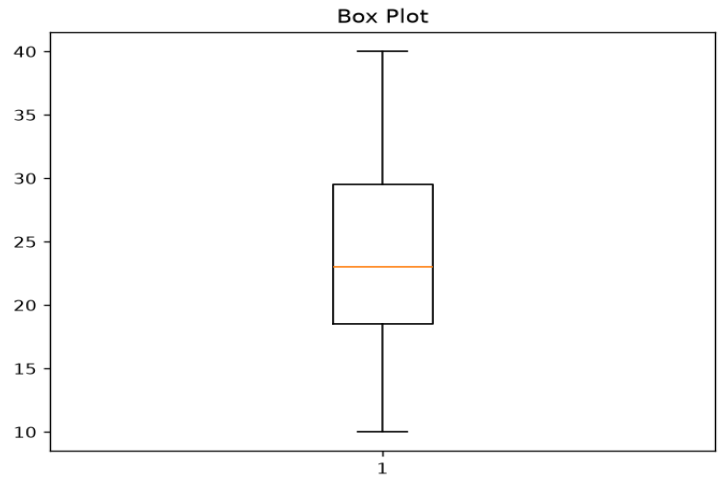
Code Snippet:

```
import matplotlib.pyplot as plt

data = [10, 15, 18, 20, 22, 24, 28, 30, 35, 40]

plt.boxplot(data)
plt.title("Box Plot")
plt.show()
```

Output:



Explanation:

The boxplot() function creates a box plot. The line inside the box represents the median of the data, while the whiskers show the range of the data. Any points outside the whiskers are called outliers.

Summary of Matplotlib Graphs:

Graph	Purpose
Line Plot	Shows trends over time
Bar Chart	Compares different categories
Histogram	Shows frequency distribution
Scatter Plot	Displays relationship between variables
Pie Chart	Shows percentage distribution
Box Plot	Shows data spread and outliers

Seaborn :

Introduction

Seaborn is a Python data visualization library that is built on top of Matplotlib. It provides a simple and attractive way to create statistical graphs. Compared to Matplotlib, Seaborn requires less code and produces better-looking graphs by default.

Seaborn works very well with Pandas DataFrames, making it a popular choice for data analysis and machine learning projects.

Features of Seaborn

Some important features of Seaborn are:

- Easy to learn and use.
- Attractive default themes and colors.
- Built on top of Matplotlib.
- Supports statistical data visualization.
- Works directly with Pandas DataFrames.
- Requires less code to create graphs.
- Useful for data analysis and machine learning.

Installation:

Install Seaborn using the following command:

```
pip install seaborn
```

Importing Seaborn:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

For the examples below, we will use Seaborn's built-in tips dataset**:**

```
tips = sns.load_dataset("tips")
```

Common Graphs in Seaborn

Seaborn provides different types of graphs for statistical visualization. Some of the commonly used graphs are discussed below.

1. Scatter Plot

Description

A scatter plot is used to display the relationship between two numerical variables.

Use Case

- Study Hours vs Marks
- Height vs Weight
- Sales vs Profit

Code Snippet:

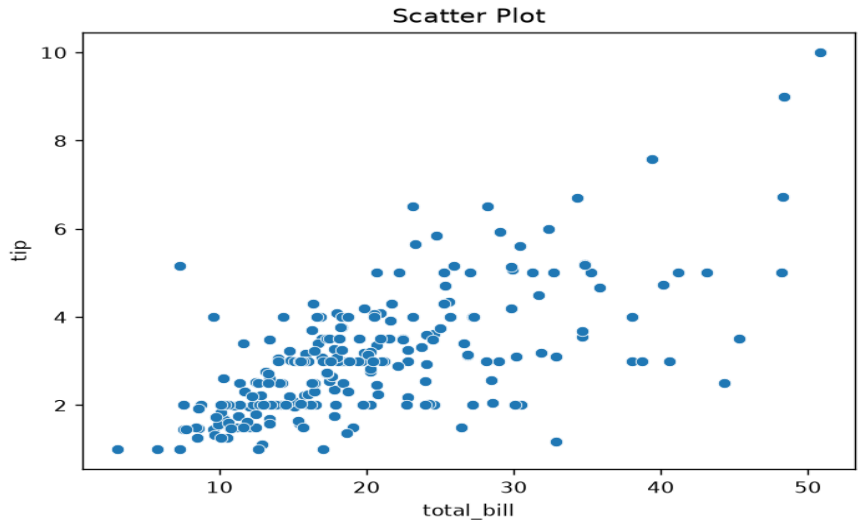
```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")

sns.scatterplot(data=tips,
                x="total_bill",
                y="tip")

plt.title("Scatter Plot")
plt.show()
```

Output:



Explanation:

The scatterplot() function is used to create a scatter plot. Each point represents one observation from the dataset.

2. Line Plot

Description

A line plot shows the relationship between two variables using connected lines.

Use Case

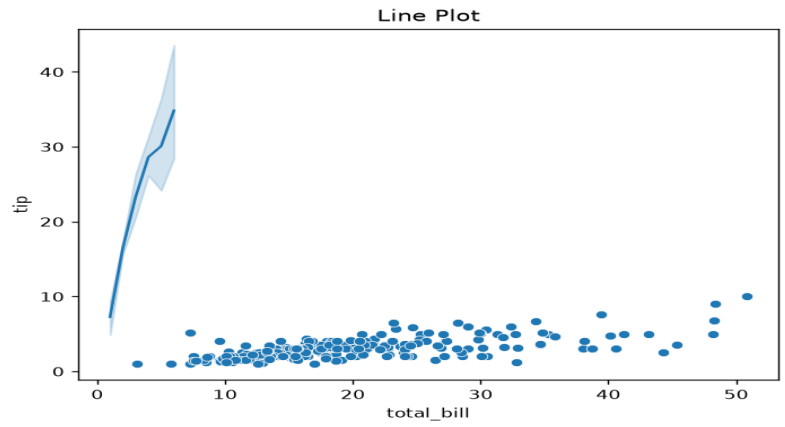
- Monthly sales
- Temperature changes
- Population growth

Code Snippet:

```
sns.lineplot(data=tips,
             x="size",
             y="total_bill")

plt.title("Line Plot")
plt.show()
```

Output:



Explanation:

The lineplot() function joins data points using lines. It is useful for identifying trends in data.

3. Bar Plot

Description

A bar plot compares numerical values across different categories.

Use Case

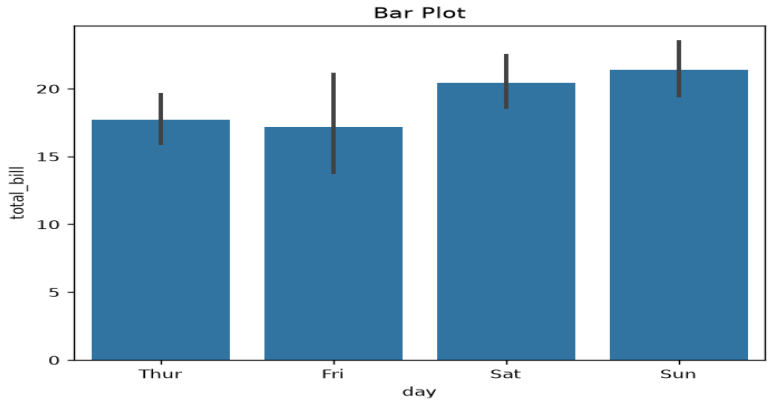
- Average sales per month
- Average marks of students
- Product comparison

Code Snippet:

```
sns.barplot(data=tips,
            x="day",
            y="total_bill")

plt.title("Bar Plot")
plt.show()
```

Output:



Explanation:

The barplot() function calculates and displays the average value for each category.

4. Count Plot

Description

A count plot shows how many observations belong to each category.

Use Case

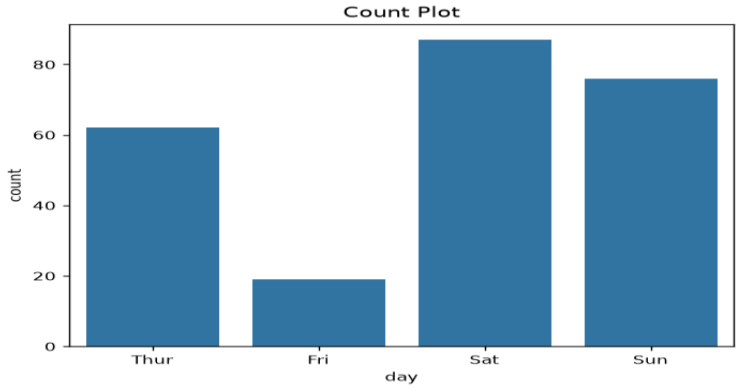
- Number of students in each class
- Number of customers by gender
- Survey responses

Code Snippet:

```
sns.countplot(data=tips,
              x="day")

plt.title("Count Plot")
plt.show()
```

Output:



Explanation:

The countplot() function counts the number of occurrences of each category automatically.

5. Histogram

Description

A histogram is used to understand how data is distributed.

Use Case

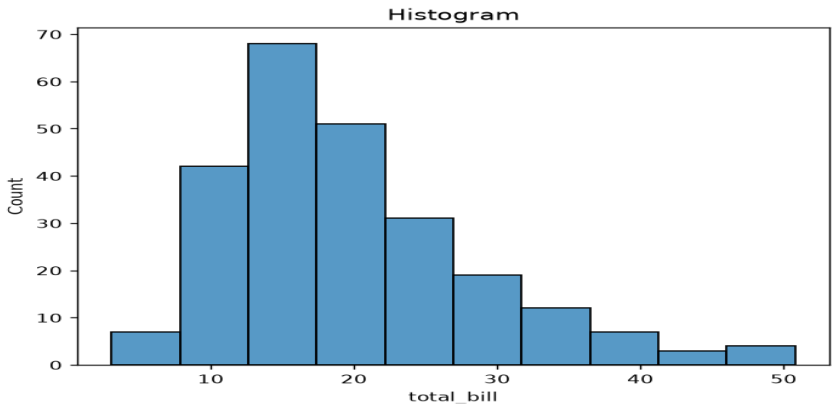
- Marks distribution
- Age distribution
- Salary analysis

Code Snippet:

```
sns.histplot(data=tips,
             x="total_bill",
             bins=10)

plt.title("Histogram")
plt.show()
```

Output:



Explanation:

The histplot() function divides the data into bins and displays the frequency of values.

6. Box Plot

Description

A box plot shows the spread of data and helps identify outliers.

Use Case

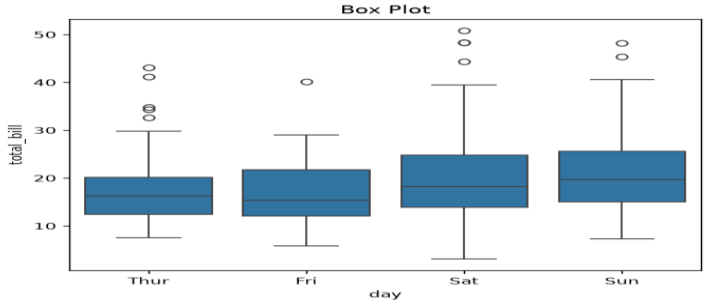
- Salary analysis
- Student marks
- Product prices

Code Snippet:

```
sns.boxplot(data=tips,
            x="day",
            y="total_bill")

plt.title("Box Plot")
plt.show()
```

Output:



Explanation:

The box represents the middle 50% of the data, while the line inside the box shows the median value.

7. Violin Plot

Description

A violin plot is similar to a box plot but also shows the density of the data.

Use Case

- Comparing data distribution
- Statistical analysis

Code Snippet:

```
sns.violinplot(data=tips,
               x="day",
               y="total_bill")

plt.title("Violin Plot")
plt.show()
```

Output:

Explanation:

The wider sections of the violin indicate where more data points are concentrated.

8. Heatmap

Description

A heatmap displays values using different colors. It is commonly used to visualize correlations between variables.

Use Case

- Correlation analysis
- Feature selection
- Data analysis

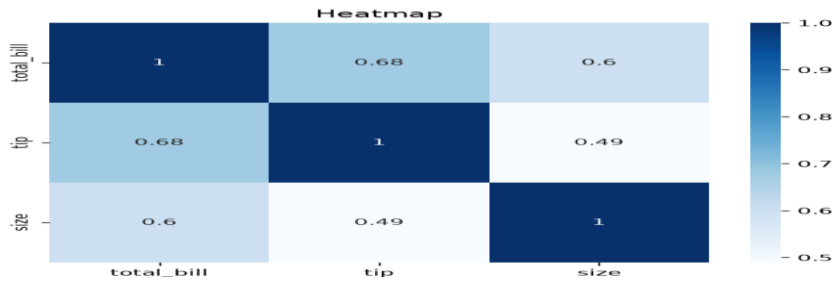
Code Snippet:

```
correlation = tips.corr(numeric_only=True)

sns.heatmap(correlation,
            annot=True,
            cmap="Blues")

plt.title("Heatmap")
plt.show()
```

Output:



Explanation:

The heatmap() function uses colors to represent values. Higher correlation values appear in darker shades.

9. Pair Plot

Description

A pair plot displays relationships among multiple numerical variables in a dataset.

Use Case

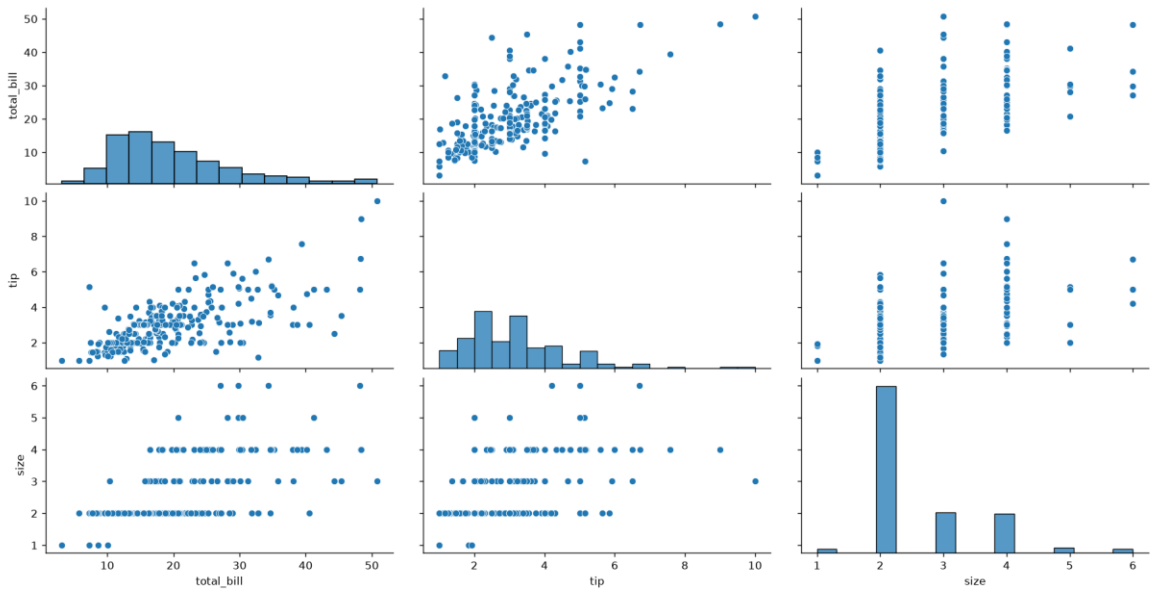
- Exploratory Data Analysis (EDA)
- Understanding relationships between variables

Code Snippet:

```
sns.pairplot(tips)

plt.show()
```

Output:



Explanation:

The pairplot() function creates scatter plots between every pair of numerical columns and histograms on the diagonal.

Comparison Between Matplotlib and Seaborn:

Both Matplotlib and Seaborn are popular Python libraries used for data visualization. Seaborn is built on top of Matplotlib, which means it uses Matplotlib internally while providing a simpler and more attractive interface for creating statistical graphs. The choice between these libraries depends on the type of visualization and the user's requirements.

Comparison Table

Feature	Matplotlib	Seaborn
Ease of Use	Easy to learn but requires more code	Very easy to use with simple syntax
Customization	Highly customizable	Good customization but less than Matplotlib
Default Appearance	Basic graphs	Attractive graphs with built-in themes
Interactivity	Limited	Limited (uses Matplotlib as backend)
Statistical Graphs	Basic support	Excellent support for statistical visualization
Performance with Large Datasets	Good performance	Good performance, but slightly slower because it is built on Matplotlib
Integration	Works well with NumPy and Pandas	Best suited for Pandas DataFrames
Best Use	General-purpose plotting and customized graphs	Statistical analysis and attractive visualizations

---

Advantages of Matplotlib

Some advantages of Matplotlib are:

- It supports many different types of graphs.
- It provides complete control over the appearance of graphs.
- It is suitable for creating publication-quality figures.
- It has a large community and detailed documentation.
- It works well with libraries such as NumPy and Pandas.

Disadvantages of Matplotlib

- Beginners may find it slightly difficult because it requires more code.
- The default graph style is simple compared to Seaborn.
- Creating complex statistical graphs can take more effort.

When Should We Use Matplotlib?

Matplotlib is a good choice when:

- We need complete control over the graph.
- We want to create customized charts.
- We are preparing reports or research papers.
- We need specialized visualizations.

Advantages of Seaborn

Some advantages of Seaborn are:

- It is easy to learn and beginner-friendly.
- It creates attractive graphs with built-in themes.
- It provides many statistical plots with very little code.
- It works directly with Pandas DataFrames.
- It is useful for Exploratory Data Analysis (EDA).

Disadvantages of Seaborn

- It provides fewer customization options than Matplotlib.
- Some advanced graph customization still requires Matplotlib.
- Since it is built on Matplotlib, it depends on Matplotlib for many functions.

When Should We Use Seaborn?

Seaborn is a better choice when:

- We want attractive graphs with less code.
- We are working with statistical data.
- We are analyzing datasets stored in Pandas DataFrames.
- We want to perform quick Exploratory Data Analysis (EDA).

Conclusion:

Data visualization is an important part of data analysis because it helps us understand information more clearly. Python provides many visualization libraries, among which Matplotlib and Seaborn are two of the most widely used.

Matplotlib is suitable for creating highly customized graphs and gives users complete control over the appearance of charts. On the other hand, Seaborn simplifies the process of creating beautiful and informative statistical graphs with fewer lines of code.

Both libraries have their own advantages and are widely used in data science, machine learning, business analysis, and research. Beginners can start with Matplotlib to understand the basics of plotting and then use Seaborn for creating more attractive and statistical visualizations.